Archived Web Pages & the Wayback Machine

Week 7 · Documents module · Textbook Chapter 7

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 28, 30 & October 2, 2026

This week at a glance

The web is not permanent. This week we scrape the *past* — and measure how pages change over time.

Monday | Concepts & notebook intro

- Web archiving, the Wayback Machine by hand, how captures break, and what researchers do with them

Wednesday | Notebook lab

- Work the Chapter 7 notebook: Availability API, CDX, tracking a Terms-of-Service document

Friday | Backlog standup

- A standup, then pairs by chapter on three activities: triage an issue, review a pull request, turn an issue into a pull request

Companion notebook

`ch-07-archives.ipynb`

Bring a laptop

Monday and Wednesday are both hands-on this week — bring a laptop both days.

1. Monday · Concepts

Link rot

38% of the web pages that existed in 2013 were gone a decade later.

Usually the website is still up — one page on it was deleted or removed.

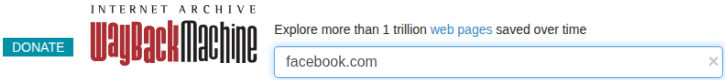
How do you study something that might not be there tomorrow?

How do you measure a *change* when you can only see today's version?



Web pages after a decade or so

Web archiving

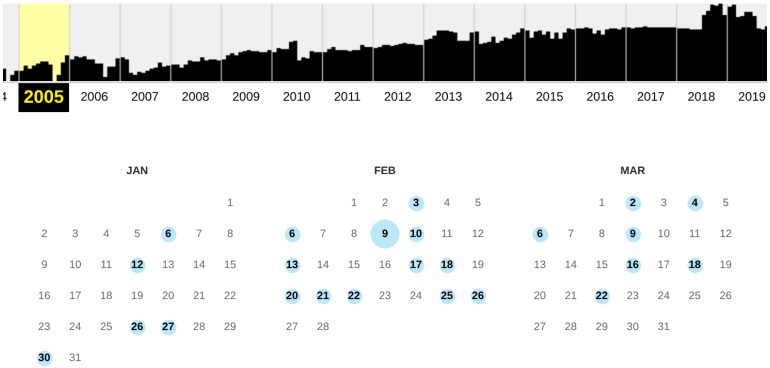


Web archiving: collect pages as they looked at a moment, store them, and let anyone replay them later.

The **Internet Archive** is a nonprofit library Brewster Kahle founded in 1996. Its **Wayback Machine** opened to the public in 2001. In October 2025 it passed **one trillion** archived web pages.

National libraries and universities archive the web too. The Wayback Machine is the one anyone can search: `https://web.archive.org`

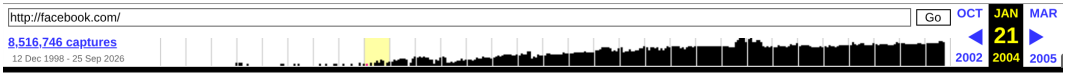
Finding a capture



Type a URL into `https://web.archive.org`. The bars across the top are the years — taller means more captures that year.

Click a year for its calendar. Each dot is a day with captures, and a bigger dot means more of them. Hover a dot to list that day's capture times.

Moving through time

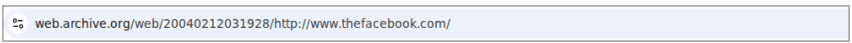


Every capture opens with this toolbar: the URL, how many times it was captured, and a strip of bars across the years.

Click anywhere on the strip to jump to that moment. The arrows and the neighboring months and years step you backward and forward.

The big yellow date is the capture on screen — here, facebook.com on January 21, 2004.

The URL is the query



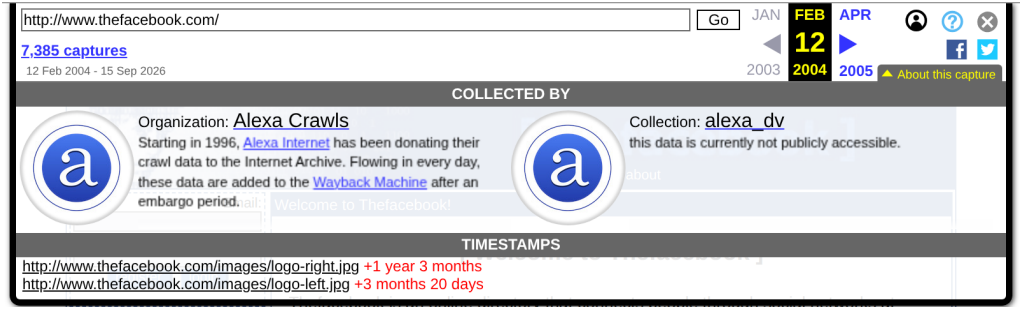
The 14 digits are a timestamp in UTC: year, month, day, hour, minute, second. `20040212031928` is February 12, 2004, at 03:19:28.

Shorten it and the Archive finds the closest capture: `web.archive.org/web/2004/thefacebook.com`

Add a star for a calendar, `/web/2005*/facebook.com`, or one at the end for every captured page on a site, `/web/*/thefacebook.com/*`

Nothing there yet? **Save Page Now** (`https://web.archive.org/save`) asks for a capture right now.

About this capture



Click **About this capture** on the toolbar. **Collected by** names the organization and crawl that saved the page. **Timestamps** lists when each image, script, and stylesheet on it was saved.

The two logos on this February 12, 2004 capture of Thefacebook were saved **three months** and **more than a year** later.

An archived page is a composite: pieces from different moments, stitched together when you replay it.

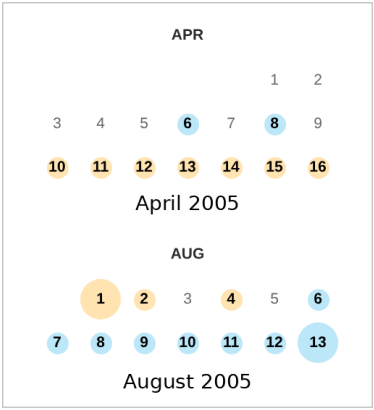
Reading the dots

A dot's color is the HTTP status code the crawler got that day (Week 5):

Blue — 2xx, the page loaded
Green — 3xx, a redirect
Orange — 4xx, like 403 Forbidden or 404 Not Found
Red — 5xx, the server failed

facebook.com in 2005: blue until April 8, then a run of orange 403s, then blue again from August 6 — now serving Thefacebook's homepage. The domain had changed hands.

Orange is not the archive failing. It faithfully recorded that the site said no.



facebook.com, 2005.

Broken captures

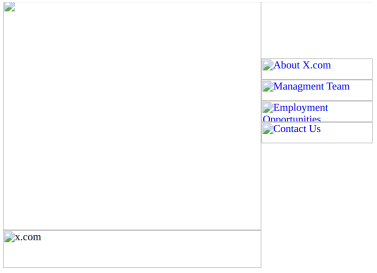
`https://web.archive.org/web/19991114081850/http://x.com/`

The HTML was saved on November 14, 1999. Its images were not.

Replay borrows each image from its nearest capture in time. Six of these seven have no good capture at all: the first ones, from August and September 2000, are all 404s.

Files served from another host — `static.example.com`, a CDN — have to be fetched on their own. Often they weren't.

URLs that JavaScript assembles while the page runs are often invisible to the crawler. Next week's problem.



x.com, November 14, 1999.

Missing captures

Sometimes the Wayback Machine "has not archived that URL." No crawler followed a link there, and nobody asked.

Subdomains are separate sites. `blog.example.com` can be missing while `example.com` has thousands of captures.

Logins and paywalls get saved as the login page.

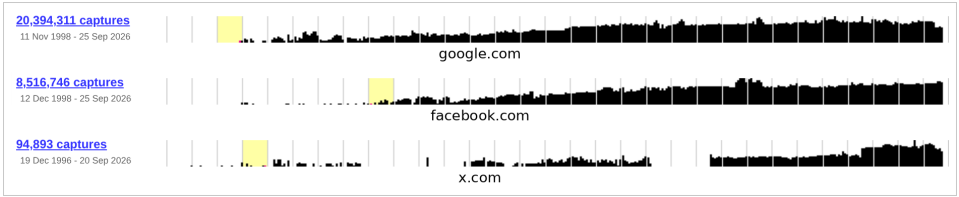
Exclusions. Site owners can request to hide their pages.

Soft 404s. A blue dot can still be an error page, if the server sent it with a 200.



Obi-Wan logging onto the Wayback Machine

Why some sites are archived better



google.com: more than 20 million captures. facebook.com: 8.5 million. x.com: 94,893. (September 2026)

Crawlers follow links, so well-linked, popular sites get found and revisited constantly. Captures also come from Save Page Now, volunteers rescuing dying sites (Archive Team), libraries (Archive-It), and special crawls like the End of Term archive of U.S. government sites at every presidential transition since 2008.

Consequences

The archive is a sample, not a census. Popular, well-linked sites are over-represented — and your sample inherits that.

Why is it so slow?

A nonprofit library serves more than a trillion archived pages, free, to anyone who asks.

It is also a target. In October 2024 a data breach and waves of DDoS attacks took the Wayback Machine offline for days.

And it is in court. It lost to book publishers in *Hachette v. Internet Archive* (2024), and settled record labels' claim of up to $621 million over its digitized 78s (2025).

Who should pay to remember the web?



web.archive.org, while making this slide.

Consequences

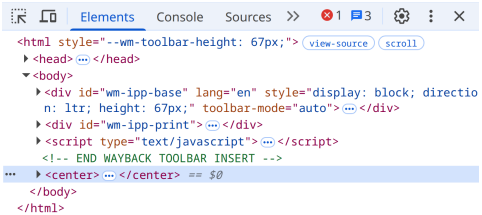
Be a polite client: pause between requests, retry with backoff when it fails, and cache what you download.

The Inspector still works

Right-click anything on an archived page and choose **Inspect**. Weeks 5 and 6 still apply: the DOM, selectors, attributes, tables.

The Archive adds a layer on top. `<div id="wm-ipp-base">` is its toolbar, and a comment marks `END WAYBACK TOOLBAR INSERT`. The original page starts right after.

Links stay inside the archive. Thefacebook's register link still carries a session ID from 2004: `register.php?PHPSESSID=df5154e5...`



Chrome DevTools on Thefacebook, February 12, 2004.

Raw or wrapped? The `id_` rule

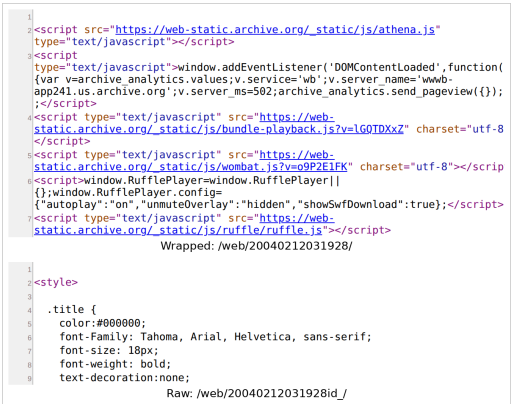
What the Inspector shows is the *wrapped* capture: the Archive's scripts in the `<head>`, its toolbar in the `<body>`, and links rewritten to stay in the archive.

Fine for browsing. Poison for measurement — your counts of links, scripts, and images include the scaffolding.

Add `id_` after the timestamp for the original bytes:

`.../web/20040212031928id_/http://www.thefacebook.com/`

Wednesday's measurement code fetches every page this way.

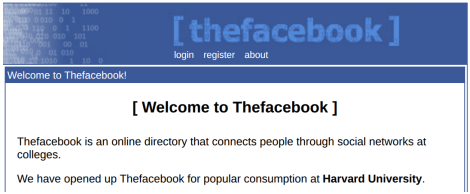


The same capture, wrapped (top) and raw (bottom).

Facebook before Facebook



facebook.com, January 21, 2004.



thefacebook.com, February 12, 2004.

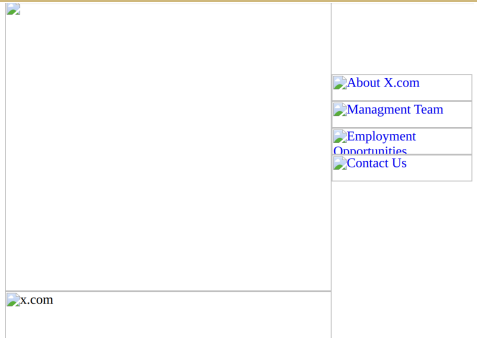
Two weeks before Thefacebook launched, facebook.com belonged to AboutFace, “The first name in directories.” It sold software to help colleges “eliminate the need for printed facebooks.”

Thefacebook bought the domain in 2005 for $200,000 and dropped the “The.”

x.com before X



x.com, April 11, 1997.

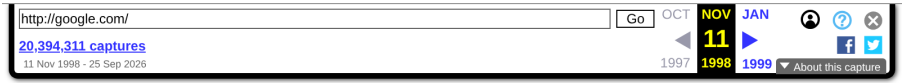


x.com, November 14, 1999.

1997: “not nearly the worst place on the web!!!” — home of Dave’s World and Rob’s House of Weather.

1999: X.com, an online bank Elon Musk co-founded. It merged with Confinity, the company behind PayPal, in 2000. Musk bought the domain back from PayPal in 2017, and X, formerly Twitter, started using it in 2023.

google.com, November 1998



The oldest capture of google.com: a heading and two links.

“Might-work-some-of-the-time-prototype that is much more up to date.”

More than twenty million captures later (September 2026).

One old page is a curiosity. Many old pages, compared systematically, are data.

- **One site, over time.** A platform's Terms of Service across twenty years — Wednesday's notebook. Amos et al. (2021) did it for over a million privacy policies from more than 130,000 websites.
- **Many sites, over time.** How third-party tracking spread across the web from 1996 to 2016 (Lerner et al., 2016).
- **Disappearance itself.** Which pages rot, how fast, and where (Pew Research Center, 2024).
- **Before and after an event.** EDGI used the Wayback Machine to document climate-change language disappearing from federal websites in 2017.

Arora et al. (2016) is a methods guide to mining websites this way.

Lerner, Kornfeld Simpson, Kohno, and Roesner (USENIX Security 2016) replayed archived pages from 1996 to 2016 and logged the third-party requests each one made.

Tracking grew in both prevalence and complexity, starting from the first tracker they found in 1996.

A big part of the paper measures where the archive's view of the past is imperfect — today's failure modes — and how to correct for it.



Closing activity: time travel with the Wayback Machine

1. Pick a site that matters to you: your high school, your hometown paper, a game, a band. Find its earliest capture
2. Find the capture closest to the day you were born
3. Find a broken capture. Use About this capture or the Inspector to explain what broke
4. Find the first capture where something changed: a logo, a slogan, a price. Open the capture halfway between two others. Repeat until you find the change
5. Find a page the Archive does not have yet. Save it with Save Page Now

Stuck? Start with thefacebook.com, x.com, google.com, or colorado.edu

Report back with your oldest find, your weirdest find, and your most broken page



Loading the oldest snapshot on Wayback

2. Wednesday · Notebook Lab

Finding a snapshot — the Availability API

The simplest question: does a snapshot exist near a date? Pass query args as a `params` dict so `requests` URL-encodes them (one value is itself a URL):

```
import requests

api_url = "https://archive.org/wayback/available"
response = requests.get(api_url, params={"url": "https://www.cnn.com",
                                         "timestamp": "20000815"}) # Aug 15, 2000
data = response.json()

if data["archived_snapshots"]:
    snap = data["archived_snapshots"]["closest"] # closest capture to that date
    print(snap["timestamp"], snap["url"])
else:
    print("No snapshot available near this date")
```

Retrieve it raw with `id_`, then measure

Rewrite the snapshot URL to the `id_` form, then parse it like any page:

```
from bs4 import BeautifulSoup

ts = snap["timestamp"]
raw_url = snap["url"].replace(ts, ts + "id_") # id_ -> original, unwrapped capture

soup = BeautifulSoup(requests.get(raw_url).text, "html.parser")
stats = {"year": ts[:4], "num_links": len(soup.find_all("a")),
         "num_scripts": len(soup.find_all("script"))}
```

The same measurement across three eras tells the web's story:

```
# year num_links text_length num_images num_scripts
# 2000 150 8500 20 2 # largely static HTML
# 2024 800 45000 120 95 # a JS app that renders
```

The whole history — the CDX Server API

Availability finds *one* snapshot; CDX enumerates *all* of them. Two params filter server-side — less data, less cleanup:

```
cdx = "https://web.archive.org/cdx/search/cdx"
params = {"url": "facebook.com/terms",
          "output": "json",
          "fl": "timestamp,original,statuscode,length",
          "filter": "statuscode:200", # only 200s
          "collapse": "digest"} # drop repeats
headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
data = requests.get(cdx, params=params, headers=headers,
                    timeout=60).json()

df = pd.DataFrame(data[1:], columns=data[0])
df["timestamp"] = pd.to_datetime(
    df["timestamp"], format="%Y%m%d%H%M%S")
```

Tracking a document over time

Wrap the Availability + `id_` + parse steps in `get_wayback_content()` (returns `None` on a miss), then loop over quarterly dates:

```
dates = pd.date_range(start="2005-01-01", end="2024-01-01", freq="QS")

results = []
for date in dates:
    snap = get_wayback_content("facebook.com/terms", date)
    if snap is not None: # skip misses instead of crashing the run
        results.append(snap)
    time.sleep(1) # rest between requests -- the Archive is a nonprofit
# Retrieved 68 snapshots from 77 time points
```

Serialize immediately — you may not be able to re-scrape it later:

```
import json
with open("fb_tos_history.json", "w") as f:
    json.dump(results, f, indent=2) # (also cache each raw capture as you go)
```

Measuring change — bag-of-words

Represent each version as a vector of word frequencies (order ignored), then compare vectors. gensim comes with week 1's `webdata`; one made before October adds it with `conda install -c conda-forge gensim`.

```
import requests
from gensim.utils import simple_preprocess
from gensim import corpora, similarities

headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
url = ("https://raw.githubusercontent.com/cuinfoscience/"
       "Web-Data-Science-Book/main/data/stopwords-en.txt") # one word per line
stop = set(requests.get(url, headers=headers).text.split())
docs = [[t for t in simple_preprocess(r["content"]) if t not in stop] for r in results]

dct = corpora.Dictionary(docs) # word -> id
corpus = [dct.doc2bow(d) for d in docs] # each version -> word-count vector
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dct))
change = [1 - index[corpus[i-1]][i] for i in range(1, len(corpus))] # 0=same, 1=all-new
```

Plotting `change` over time gives a change timeline: spikes flag major revisions — a new policy, a redesign, a response to controversy.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Select a policy document. Get 5 or more versions from different dates. Plot the length of each version.
2. Use the CDX API. Compare the capture frequency and the page size of two competing organizations.
3. Calculate the similarity between all pairs of versions. Make a heatmap. Identify the points of change.
4. Examine the archive history of a government site. Report the gaps, the coverage, and the change in size.
5. Compare the earliest snapshot with the most recent snapshot. Write 200 words about the differences.
6. **5617**: assess the limits of the Wayback Machine. Include the toolbar, the capture bias, and the gaps. Compare your results with Rogers (2019)

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- a change you found in the archived past
- a question for a research design

3. Friday · Backlog Standup

Daily note questions

- **Web scraping from Safari and Chrome?** — The inspect tab should be present on all desktop web browser
- **Past student projects?** — Go poke around `github.com/cuinfoscience/`
- **Why do some timestamp return and others don't?** — Depends if Wayback blocks a request, mostly random!
- **Why does this stuff break with lots of requests?** — Wayback pretty rickety, would need to run it all myself
- **Best ways to get around 429 errors?** — Slow down, try a different time of day, use a different network

Find your chapter

51 open PRs and **53** issues going back to August 28.

Chapter	Pull requests	Issues	Pairs
2 · Ethics	6	17	3
3 · The post-API age	3	11	2
4 · Data formats	12	5	2
5 · Protocols	14	7	2–3
6 · Static pages	11	9	2



Three activities

Triage an issue

Easy. Check an issue against the book. Already fixed? A duplicate? Still a problem? Leave one comment.

`triage-an-issue.pdf`

Review a pull request

Medium. Read the change and its checks. Comment on lines, then submit a review with a verdict.

`review-a-pull-request.pdf`

Issue → pull request

Hard. Claim an issue, fix it in GitHub's web editor, and open a pull request that closes it.

`issue-to-pull-request.pdf`

Each handout goes step by step. They are in `handouts/week-07/` in the course repository.

On the boards, #15 → #59? asks: does pull request #59 fix issue #15?

Triage

- #13, #25, #39: one problem?
- #15 → #59?
- #17 → #56?
- #18 → #54?
- #28 → #63?
- #43 → #50, #79?

Review

- #50 and #79: the same paragraph. Keep one.
- #54 links for the legal cases
- #56 a Missing Manual link
- #59 the User-Agent example's output
- #63 comparing robots.txt files

Issue → pull request

- #23 formatting
- #24 Common Issues
- #26 the User-Agent
- #27 test cases
- #34 state privacy laws
- #40 replacement reminders
- #42 Step 5's `NameError`
- #44 an undefined variable
- #49 Windows

Triage

- #55 and #67: both Step 5. One problem?
- #58: no title. Suggest one.
- #60 → #80?
- #68 → #95?

Review

- #62 edits only the notebook
- #80 `probe()`'s mapping as a dictionary
- #95 a link for A/B testing

Issue → pull request

- #57 Meta's 400, never shown
- #61 the User-Agent
- #66 `autopsy()` and an expired certificate
- #69 the steps' wording
- #70 good and bad APIs
- #85 misconceptions

Triage

- #90 → #92?
- #97: no title. Suggest one.

Review

- #92 and #99: neighboring rows of one table
- #81 XML vs. JSON
- #82 a W3C link
- #86 Step 5
- #98 JSON object order
- #100 JSON and Python types
- #101 Kubernetes formats
- For me: #88, #89, #93, #102 conflict with `main`

Issue → pull request

- #74 HTML as a format
- #83 `pubDate` as a date
- #189 Matplotlib and nested JSON

Triage

- #105, #110 → #194?
- #175 → #122?
- #190 → #121?
- #115: no title. Suggest one.

Review

- #113 and #127: the same exercise steps
- #104 a Missing Manual link
- #107 a DNS tip
- #108 edits only the notebook
- #109 a dash
- #116 `Retry-After`
- #118 `time.sleep()`
- #121 `dnspython`
- #122 the redirect's destination
- #126 a timeout
- For me: #106, #111, #112 edit Scapy, which #194 removes

Issue → pull request

- #114 building URLs
- #133 a DNS link

Triage

- #186 → #173?
- #187 → #131?

Review

- #130 and #188: the same Wikipedia 403 fix. Keep one.
- #131 defines `responsible_get`
- #173 the spectrum table and §6.8
- #174 Further Reading
- #178 a Chapter 4 refresher
- #179 fetch Rotten Tomatoes first
- #180 parsing cached HTML
- #181 IMDb's 202
- #182 edits only the notebook
- #183 Strategy 2's table
- For me: #171, whose head is `main`

Issue → pull request

- #134 BeautifulSoup's methods
- #172 `read_html` and `colspan`
- #176 robots.txt with `HEADERS`
- #177 CSS selectors
- #184 data attributes
- #185 a check before parsing

Chapters 1, 7, 11, and the whole book

Triage

- #19 and #29: the same ask? (chapter 1)
- #64: one folder per chapter? A question for me.

Review

- #191 and #192: the Availability API's 429s (chapter 7). Read them together.
- #78 a Census link (chapter 11)
- For me: #52 conflicts with `main` (chapter 1)

Issue → pull request

- #22 a command-line cheat sheet (an appendix)

The standup

One minute per pair, in chapter order:

Done — what merged, closed, or got a review since last Friday.

Doing — your activity, and the numbers you take from your chapter's board.

Blocked — anything only I can answer.

Leave these to me

- **Merge** and **Close**: I do both after class.
- *This branch has conflicts*: I resolve conflicts.

After class

I merge the approved pull requests with merge commits. The book republishes, and the issues they fix get closed.

I regenerate the notebooks once, so Notebook sync goes green again.

Next week

Dynamic pages: when the data appears only after JavaScript runs (Selenium).

Key takeaways

1. The web is impermanent (link rot); the **Internet Archive** is public infrastructure that preserves it.
2. **Availability API** finds one snapshot near a date; the **CDX Server** enumerates the full history — filter and collapse server-side.
3. Append `id_` for the **raw** capture; the toolbar-wrapped version pollutes every measurement.
4. Wayback + text analysis turns the archive into **longitudinal data** — measure change, don't just observe it.
5. Be a good citizen: `sleep` between requests, cache raw captures, and **serialize** — you may not be able to re-scrape.